OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Шаг валидации в бизнес-цепочке

otus.ru

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	Архитек-тура	DevOps		
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Консуль-тация	Защита				

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План занятия

- Общие сведения о валидации
- Способы валидации
- Валидация в приложении
- Особенности фреймворков
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Строить логику валидации в бэкенд-приложении
2. Применять CoR-шаблон для валидации
3. Понять как организовать валидацию средствами DSL

Смысл

Зачем вам это уметь

1. Валидация позволяет избежать логических ошибок или компрометации приложения
2. CoR позволяет сделать шаг валидации логичным и стройным
3. DSL позволяет сделать шаг валидации еще более легко читаемым

Общие сведения

Общие сведения о валидации

Особенности:

1. Любые входящие данные потенциально скомпрометированы
2. Источники для валидации:
   a. Все запросы
   b. Сообщения из MQ
   c. Ответы сторонних служб
   d. Конфигурации
   e. База данных (очень редко)

Корректная реакция

... На невалидные данные:

1. Отдать **клиенту** корректное сообщение
   a. Не содержит стектрейса
   b. Содержит исчерпывающее для пользователя описание ошибки
   c. Содержит технические данные для обнаружения (запрос, поле и пр.)
   d. По возможности, способы исправления (“возможно вы имели в виду...”)
2. Положить **в лог** все необходимые сведения, включая стактрейс
3. Настройка мониторинга:
   a. Обнаружение аномалий
   b. Обнаружение атак
   c. Обнаружение багов

Виды валидации

Виды валидации:

1. Форматный контроль
   a. Зависит от выбранного API
   b. Выполняется на входе данных до преобразования во внутренние модели
2. Логический контроль
   a. Не зависит от реализации API
   b. Характеризует логические ошибки в данных (год рождения в будущем или “столько не живут”)
   c. Выполняется в бизнес-логике приложения

Способы валидации

Форматный контроль: средствами фреймворка

Особенности:

a. Все фреймворки имеют валидационные возможности
b. Плохо управляются
c. Разбросаны по коду
d. Исключены из общей логики
e. Обработка через перехват исключений

Hibernate validator:

```
class SomeInfo(
    @field:NotNull
    @field:Pattern(regexp = Constraints.EMAIL_REGEX)
    var value: String
)
```

Форматный контроль: тихий мапер

Особенности:

a. Очень простой
b. Теряем информацию об ошибках при мапинге во внутренние модели
c. Не можем передать ошибки в цепочку

```
private fun AdCreateObject.toInternal(): MkplAd =
    MkplAd(
        title = this.title ?: "",
        description = this.description ?: "",
        adType = this.adType.fromTransport(),
        visibility = this.visibility.fromTransport(),
    )
```

Форматный контроль: специальный CoR

Особенности:

a. Посложнее
b. Гибкий
c. Мощная логика валидации
d. Отдельный контекст:
    i. Содержит внутренние и транспортные модели
    ii. Отдельный для каждого мапера

```
validation {
    validateRuleOne("...")
    validateRuleTwo("...")
    finishValidation("...")
}
```

Форматный контроль: библиотека валидации

Особенности:

a. Отдельный класс для любой валидации
b. Переиспользование валидаторов
c. Мощная логика валидации

https://www.konform.io/

```
val objValidator = Validation<SomeObject> {
    SomeObject::userId {
        pattern("^[0-9a-zA-Z_-]{1,64}\$")
    }
    SomeObject::dob {
        minAge(15)
    }
}
```

Логический контроль: часть цепочки бизнес-логики

Особенности:

a. Реализуется как часть бизнес-логики
b. Обеспечивает как валидацию, так и преобразование
c. Переиспользование валидаторов
d. Неограниченные возможности кастомизации

```
validation {
    worker("Копируем поля в adValidating") {
        adValidating = adRequest.deepCopy()
    }
    worker("Очистка id") {
        adValidating.id = MkplAdId(adValidating.id.asString().trim())
    }
    validateIdNotEmpty("Проверка на непустой id")
    validateIdProperFormat("Проверка формата id")
    finishAdValidation("Успешное завершение процедуры валидации")
}
```

Обработка ошибок

Порядок обработки ошибок: сбор информации об ошибке и логирование

1. **Для клиента:** собираем всю информацию, складываем в контекст
2. **Для админа:** Отправляем логи в месте обнаружения ошибки с расширенной информацией с уровнем логирования:
    - Debug - если ошибка предусмотрена логикой
    - Error - нештатная ситуация
3. Штатное логирование ответа с уровнем Info содержит также ошибки

```
catch (e: Throwable) {
    logger.doWithLogging( id = "${logId}-failure") {
        command?.also { ctx.command = it }
        logger.error(
            msg = "$command handling failed",
        )
        ctx.state = MkplState.FAILING
        ctx.errors.add(e.asMkplError())
        processor.exec(ctx)
        respond(ctx.toTransportAd())
    }
}
```

LIVE

Рефлексия

Основные тезисы

1.	Валидация обязательна для всех входящих данных
2.	Валидация форматная, логическая
3.	Типичные валидации: пустое поле, недопустимые символы, наличие контента и пр.
4.	Форматная валидация в маперах
5.	Логическая валидация в бизнес-логике

Домашнее задание

1. Сделать модуль бизнес-логики для своего выпускного проекта
2. Реализовать заглушки для запросов с помощью шаблона CoR
3. Выполнить валидацию в модуле бизнес-логики
4. Сделать автотесты для бизнес-логики

Срок: 26.09.2025

Рефлексия

По какому вопросу захотелось глубже изучить информацию?

Понимаете ли вы, как применять на практике то, что узнали на вебинаре. Если да, то как?

Цели вебинара

Проверка достижения целей

1. Строить логику валидации в бэкенд-приложении
2. Применять CoR-шаблон для валидации
3. Понять как организовать валидацию средствами DSL

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/otuskotlin-user
- https://github.com/otuskotlin/202312-ok-marketplace
- https://www.konform.io/
- https://hibernate.org/validator/

Следующий вебинар

26.09.2025

Модуль 7: Работа с базами данных

Практические принципы работы с БД

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары